ALGORITHM OF UNIVERSITY MANAGEMENT SYSTEM

1. Main Algorithm of University Management System

Step 1: Start the program.

Step 2: Create a Scanner object to take input from the user.

Step 3: Create a University object with the name "COMSATS University Islamabad, Wah Campus".

Step 4: Create the following office staff objects:

- Admission Office.
- Account Office.
- Exam Office.
- Departmental Staff.

Step 5: Get the Computer Science, Electrical Engineering, and Management Science departments from the university.

Step 6: Create a Departmental Staff object for the Computer Science department.

Step 7: Add the Admission Office, Account Office, Exam Office, Departmental Staff, and three Faculty objects to the university's people list.

Step 8: Create three Faculty objects and assign their respective departments and designations.

Step 9: Create three Course objects:

- CS101: Programming Fundamentals, 3 credit hours, Undergraduate level.
- CS102: Object Oriented Programming, 3 credit hours by default, Undergraduate level.
- CS103: Machine Learning, 3 credit hours by default, Graduate level.

Step 10: Assign teachers to the courses.

Step 11: Add all three courses to the university.

Step 12: Schedule the courses in the following rooms and time slots:

- CS101 in room G-01 at Mon 09:00.
- CS102 in room G-01 at Mon 11:00.
- CS103 in room Lab-01 at Tue 14:00.

Algorithm

Step 13: Create two Undergraduate Student objects and one Graduate Student object.

Step 14: Admit all three students through the Admission Office.

Step 15: Enroll the first undergraduate student in CS101 and CS102.

Step 16: Enroll the second undergraduate student in CS102.

Step 17: Enroll the graduate student in CS103.

Step 18: Record marks of 88 in CS101 and 76 in CS102 for the first undergraduate student.

Step 19: Display the welcome message.

Step 20: Initialize the menu choice to -1.

Step 21: Repeat the following steps while the user's choice is not 0:

1. Display the university name.
2. Display the menu options.
3. Take the user's choice as input.
4. Execute the corresponding operation using a switch statement.
5. Display the result of the selected operation.
6. Repeat the menu.

Step 22: If the user selects 0, display "Goodbye!" and terminate the menu loop.

Step 23: Close the Scanner object.

Step 24: End the program.

---

2. Algorithm of Person Class

Purpose: To store common information about students, faculty members, and staff.

Step 1: Initialize the static variable nextId to 1.

Step 2: Declare the following private variables:

- id
- name
- email

Algorithm

Step 3: If the no-argument constructor is called, call the parameterized constructor using this() with the default name "Unknown" and email "unknown@cui.edu".

Step 4: In the parameterized constructor:

1. Assign the current value of nextId to id.
2. Increment nextId by 1.
3. Assign the given name to name.
4. Assign the given email to email.

Step 5: Provide getter methods to return the ID, name, and email.

Step 6: Provide setter methods to update the name and email.

Step 7: In the static method getTotalPeople(), return nextId - 1.

Step 8: In the method getRole(), return "Person".

Step 9: In the toString() method, return the role, ID, name, and email in the following format:

Role #ID | Name | Email

Step 10: In the equals() method:

1. Check whether the given object is an instance of Person.
2. If true, convert it to a Person object.
3. Compare the IDs of the two objects.
4. Return true if the IDs are equal; otherwise, return false.

Step 11: End the class.

---

3. Algorithm of Student Class

Purpose: To store student information, manage course enrollment, record marks, calculate results, and manage fees.

Step 1: Extend the Person class.

Step 2: Declare the following variables:

- dept to store the student's department.
- courses as a Course array of size 6.

Algorithm

- marks as a double array of size 6.
- courseCount initialized to 0.
- feePaid initialized to 0.
- admissionDate to store the admission date.

Step 3: In the constructor:

1. Call the Person constructor using super(name, email).
2. Assign the given department to dept.
3. Set admissionDate to the current date.

Step 4: Define the default methods:

- getMaxCourses() returns 5.
- getFeePerCourse() returns 20000.
- getLevel() returns "Student".

Step 5: Override getRole() to return the student's level followed by " Student".

Step 6: Provide getter methods for the department, admission date, course count, course at a given index, and mark at a given index.

3.1 Algorithm of hasCourse() Method

Step 1: Receive a Course object as input.

Step 2: Initialize a loop from 0 to courseCount - 1.

Step 3: Compare the course at the current index with the given course using the equals() method.

Step 4: If both courses are equal, return true.

Step 5: If no matching course is found, return false.

3.2 Algorithm of enroll() Method

Step 1: Receive a Course object as input.

Step 2: Compare the student's level with the course level.

Step 3: If the levels are different, return a failure message containing the student's name, student level, course code, and course level.

Algorithm

Step 4: Call hasCourse() to check whether the student is already enrolled in the course.

Step 5: If the student is already enrolled, return a failure message.

Step 6: Check whether courseCount is greater than or equal to getMaxCourses().

Step 7: If the maximum number of courses has been reached, return a failure message.

Step 8: Call isFull() on the course.

Step 9: If the course is full, return a failure message.

Step 10: If all conditions are satisfied:

1. Store the course in courses[courseCount].
2. Set marks[courseCount] to 0.
3. Increment courseCount by 1.
4. Call addStudent() on the course to increment its enrolled student count.

Step 11: Return a success message containing the student's name and course code.

Step 12: End the method.

3.3 Algorithm of setMark() Method

Step 1: Receive a Course object and a mark as input.

Step 2: Check whether the mark is less than 0 or greater than 100.

Step 3: If the mark is outside this range, return false.

Step 4: Search the student's enrolled courses from index 0 to courseCount - 1.

Step 5: If the given course matches an enrolled course:

1. Store the mark at the corresponding index in the marks array.
2. Return true.

Step 6: If the course is not found, return false.

3.4 Algorithm of getAverage() Method

Step 1: Check whether courseCount is 0.

Step 2: If no courses are enrolled, return 0.

Step 3: Initialize total to 0.

Algorithm

Step 4: Loop through all enrolled courses.

Step 5: Add each course mark to total.

Step 6: Divide total by courseCount.

Step 7: Return the calculated average.

3.5 Algorithm of getGrade() Method

Step 1: Call getAverage() to calculate the average marks.

Step 2: Check the average according to the following conditions:

- If the average is greater than or equal to 85, return grade A.
- Otherwise, if the average is greater than or equal to 70, return grade B.
- Otherwise, if the average is greater than or equal to 60, return grade C.
- Otherwise, if the average is greater than or equal to 50, return grade D.
- Otherwise, return grade F.

Step 3: End the method.

3.6 Algorithm of Fee Methods

Step 1: In getTotalFee(), multiply courseCount by getFeePerCourse() and return the result.

Step 2: In getFeePaid(), return the total amount already paid.

Step 3: In getFeeDue(), subtract feePaid from getTotalFee() and return the result.

Step 4: In payFee(amount), add the given amount to feePaid.

Step 5: In toString(), call the superclass toString() and append the department name and current course count with the maximum course limit.

Step 6: End the class.

---

4. Algorithm of UndergraduateStudent Class

Purpose: To represent undergraduate students with specific course and fee limits.

Step 1: Extend the Student class.

Step 2: In the constructor, receive the student's name, email, and department.

Algorithm

Step 3: Call the Student constructor using super(name, email, dept).

Step 4: Override getMaxCourses() to return 6.

Step 5: Override getFeePerCourse() to return 15000.

Step 6: Override getLevel() to return "Undergraduate".

Step 7: End the class.

---

5. Algorithm of GraduateStudent Class

Purpose: To represent graduate students and store their thesis topics.

Step 1: Extend the Student class.

Step 2: Declare a private variable thesisTopic.

Step 3: In the constructor, receive the student's name, email, department, and thesis topic.

Step 4: Call the Student constructor using super(name, email, dept).

Step 5: Assign the given thesis topic to thesisTopic.

Step 6: In getThesisTopic(), return the thesis topic.

Step 7: Override getMaxCourses() to return 4.

Step 8: Override getFeePerCourse() to return 25000.

Step 9: Override getLevel() to return "Graduate".

Step 10: Override toString() to call the Student class's toString() method and append the thesis topic.

Step 11: End the class.

---

6. Algorithm of Faculty Class

Purpose: To store faculty information, including designation and department.

Step 1: Extend the Person class.

Step 2: Declare the variables designation and dept.

Step 3: In the constructor:

1. Call super(name, email).
2. Assign the given department to dept.
3. Assign the given designation to designation.

Step 4: Provide getter methods for department and designation.

Step 5: Override getRole() to return "Faculty".

Step 6: Override toString() to append the designation and department name to the superclass's string representation.

Step 7: End the class.

---

7. Algorithm of Staff Class

Purpose: To represent university staff members and their office responsibilities.

Step 1: Extend the Person class.

Step 2: Declare a private variable office.

Step 3: In the constructor:

1. Receive the name, email, and office.
2. Call super(name, email).
3. Assign the given office to office.

Step 4: In getOffice(), return the office name.

Step 5: In performDuty(), return "General staff duties.".

Step 6: Override getRole() to return the office name followed by " Staff".

Step 7: End the class.

---

8. Algorithm of AdmissionOffice Class

Purpose: To admit students to the university.

Step 1: Extend the Staff class.

Step 2: In the constructor, receive the name and email.

Algorithm

Step 3: Call super(name, email, "Admission Office").

8.1 Algorithm of admit() Method

Step 1: Receive a University object and a Student object.

Step 2: Call uni.addStudent(s) to add the student to the university.

Step 3: If addStudent() returns false, return the message "Failed: the university cannot take more students.".

Step 4: If the student is added successfully, call addStudent() on the student's department to increment its student count.

Step 5: Return a message containing:

- Student name.
- Department name.
- Student level.
- Student ID.
- Admission date.

Step 6: End the method.

Step 7: Override performDuty() to return "Admits new students and gives them an ID.".

Step 8: End the class.

---

9. Algorithm of AccountOffice Class

Purpose: To collect student fees and generate payment challans.

Step 1: Extend the Staff class.

Step 2: In the constructor, receive the name and email.

Step 3: Call super(name, email, "Account Office").

9.1 Algorithm of collectFee(Student, amount) Method

Step 1: Receive a Student object and a payment amount.

Step 2: Check whether the amount is less than or equal to 0.

Step 3: If true, return a failure message stating that the amount must be greater than 0.

Algorithm

Step 4: Compare the payment amount with the student's outstanding fee.

Step 5: If the amount is greater than the fee due, return a failure message displaying the fee due.

Step 6: If the amount is valid:

1. Call payFee(amount) on the student.
2. Create a Random object.
3. Generate a random challan number between 1000 and 9999 using 1000 + r.nextInt(9000).
4. Create a new Date object to obtain the current date and time.

Step 7: Return the challan number, current date and time, amount received, student name, and remaining fee due.

Step 8: End the method.

9.2 Algorithm of collectFee(Student) Method

Step 1: Receive a Student object.

Step 2: Call getFeeDue() to determine the outstanding fee.

Step 3: If the fee due is less than or equal to 0, return a message stating that nothing is due.

Step 4: Otherwise, call the overloaded collectFee(Student, amount) method, passing the full outstanding fee as the amount.

Step 5: Return the result of that method.

Step 6: Override performDuty() to return "Collects fees and keeps the payment records.".

Step 7: End the class.

---

10. Algorithm of ExamOffice Class

Purpose: To record student marks and prepare results.

Step 1: Extend the Staff class.

Step 2: In the constructor, receive the name and email.

Step 3: Call super(name, email, "Exam Office").

Algorithm

Algorithm of recordMarks() Method

Step 1: Receive a Student object, a Course object, and marks.

Step 2: Call setMark(c, marks) on the student.

Step 3: If setMark() returns true, return a success message containing the student's name, marks, and course code.

Step 4: Otherwise, return a failure message stating that the student is not enrolled in the course or the marks are outside the range 0 to 100.

Step 5: Override performDuty() to return "Records marks and prepares results.".

Step 6: End the class.

---

11. Algorithm of DepartmentalStaff Class

Purpose: To manage department-related classroom scheduling.

Step 1: Extend the Staff class.

Step 2: Declare a private variable dept.

Step 3: In the constructor:

1. Receive the name, email, and department.
2. Call super(name, email, "Departmental Support").
3. Assign the given department to dept.

Algorithm of scheduleClass() Method

Step 1: Receive a University object, a Course object, a Room object, and a time slot.

Step 2: Call uni.isRoomFree(r, slot) to check whether the room is available at the specified time.

Step 3: If the room is not free, return a failure message containing the room number and time slot.

Step 4: If the room is free, call c.setSchedule(r, slot) to assign the room and time slot to the course.

Step 5: Return a success message containing the course code, room number, and time slot.

Algorithm

Step 6: Override performDuty() to return a message stating that the staff member helps the department with schedules and classrooms.

Step 7: End the class.

---

12. Algorithm of Room Class

Purpose: To store classroom information.

Step 1: Declare two private final variables:

- roomNo
- capacity

Step 2: In the constructor, receive the room number and capacity.

Step 3: Assign the received values to the respective variables.

Step 4: Provide getRoomNo() to return the room number.

Step 5: Provide getCapacity() to return the room capacity.

Step 6: Override toString() to return the room number and capacity in the format:

Room Number (capacity Capacity)

Step 7: End the class.

---

13. Algorithm of Course Class

Purpose: To store course details, teacher information, room scheduling, and enrollment count.

Step 1: Declare the following variables:

- code
- name
- creditHours
- level
- teacher
- room

Algorithm

- slot, initialized to "Not scheduled".
- enrolledCount, initialized to 0.

Step 2: In the first constructor, receive the course code, course name, credit hours, and level.

Step 3: Assign the received values to the respective variables.

Step 4: In the second constructor, receive the course code, course name, and level.

Step 5: Call the first constructor using this(code, name, 3, level) to set the default credit hours to 3.

Step 6: Provide getter methods for course code, course name, course level, room, and time slot.

Step 7: In setTeacher(teacher), assign the given Faculty object to the course's teacher variable.

Step 8: In setSchedule(room, slot), assign the given room and time slot to the course.

13.1 Algorithm of isFull() Method

Step 1: Check whether the course's room is not null.

Step 2: If the room is not null, compare enrolledCount with the room capacity.

Step 3: Return true if enrolledCount is greater than or equal to the room capacity.

Step 4: Otherwise, return false.

13.2 Algorithm of addStudent() Method

Step 1: Increment enrolledCount by 1.

Step 2: End the method.

13.3 Algorithm of equals() Method

Step 1: Receive an object.

Step 2: Check whether the object is an instance of Course.

Step 3: If true, convert it to a Course object.

Step 4: Compare the course codes of both objects.

Step 5: Return true if the course codes are equal; otherwise, return false.

Step 6: If the object is not an instance of Course, return false.

13.4 Algorithm of toString() Method

Algorithm

Step 1: Initialize teacherText to "No teacher".

Step 2: If the teacher is not null, assign the teacher's name to teacherText.

Step 3: Initialize roomText to "No room".

Step 4: If the room is not null, assign the room number to roomText.

Step 5: Return the course code, course name, level, credit hours, teacher name, room number, time slot, and enrolled student count.

Step 6: End the class.

---

14. Algorithm of Department Class

Purpose: To store department information and maintain its student count.

Step 1: Declare the variables name and studentCount.

Step 2: Initialize studentCount to 0.

Step 3: In the constructor, receive the department name and assign it to name.

Step 4: In getName(), return the department name.

Step 5: In getStudentCount(), return the student count.

Step 6: In addStudent(), increment studentCount by 1.

Step 7: End the class.

---

15. Algorithm of University Class

Purpose: To manage departments, rooms, courses, students, and people.

Step 1: Declare the following variables:

- name to store the university name.
- depts as a Department array of size 4.
- rooms as a Room array of size 5.
- courses as a Course array of size 10.
- students as a Student array of size 50.

Algorithm

- people as a Person array of size 100.
- courseCount, initialized to 0.
- studentCount, initialized to 0.
- peopleCount, initialized to 0.

Step 2: In the constructor, receive the university name and assign it to name.

Step 3: Create and store four departments:

1. Computer Science.
2. Electrical Engineering.
3. Management Science.
4. Civil Engineering.

Step 4: Create and store five rooms:

1. G-01 with capacity 40.
2. G-02 with capacity 35.
3. G-03 with capacity 30.
4. G-04 with capacity 30.
5. Lab-01 with capacity 25.

Step 5: Provide getter methods for the university name, department count, department at a given index, room count, room at a given index, course count, course at a given index, people count, and person at a given index.

15.1 Algorithm of addCourse() Method

Step 1: Receive a Course object.

Step 2: Store the course in courses[courseCount].

Step 3: Increment courseCount by 1.

Step 4: End the method.

15.2 Algorithm of addPerson() Method

Step 1: Receive a Person object.

Step 2: Store the object in people[peopleCount].

Algorithm

Step 3: Increment peopleCount by 1.

Step 4: End the method.

15.3 Algorithm of addStudent() Method

Step 1: Receive a Student object.

Step 2: Check whether studentCount is greater than or equal to the size of the students array or peopleCount is greater than or equal to the size of the people array.

Step 3: If either condition is true, return false.

Step 4: Otherwise:

1. Store the student in students[studentCount].
2. Increment studentCount by 1.
3. Call addPerson(s) to add the student to the people array.

Step 5: Return true.

Step 6: End the method.

15.4 Algorithm of findStudent() Method

Step 1: Receive a student ID.

Step 2: Search the students array from index 0 to studentCount - 1.

Step 3: Compare each student's ID with the given ID.

Step 4: If a matching ID is found, return the corresponding Student object.

Step 5: If no matching student is found, return null.

Step 6: End the method.

15.5 Algorithm of findCourse() Method

Step 1: Receive a course code.

Step 2: Search the courses array from index 0 to courseCount - 1.

Step 3: Compare each course code with the given code using equalsIgnoreCase().

Step 4: If a match is found, return the corresponding Course object.

Step 5: If no matching course is found, return null.

Algorithm

Step 6: End the method.

15.6 Algorithm of isRoomFree() Method

Step 1: Receive a Room object and a time slot.

Step 2: Search all courses stored in the university.

Step 3: For each course, check whether its room reference is the same as the given room reference and its time slot equals the given time slot.

Step 4: If both conditions are true for any course, return false.

Step 5: If no course is using the same room at the same time, return true.

Step 6: End the class.

---

16. Algorithm of Runner Class and Menu Operations

16.1 Algorithm of Menu Display

Step 1: Create a Scanner object.

Step 2: Create and initialize the university, office staff, departments, faculty, courses, classrooms, and sample students.

Step 3: Add the office staff and faculty to the university's people list.

Step 4: Add the courses to the university and assign teachers.

Step 5: Schedule the sample courses.

Step 6: Admit the sample students.

Step 7: Enroll the sample students in their respective courses.

Step 8: Record the sample marks.

Step 9: Display the welcome message.

Step 10: Set choice = -1.

Step 11: While choice != 0:

1. Create a character array of size 45.
2. Fill every position of the array with the = character.
3. Display the separator line.

Algorithm

4. Display the university name.
5. Display the separator line again.
6. Display the following menu:

- 1. View departments.
- 2. View courses.
- 3. View all people (students, faculty, staff).
- 4. Admit a new student.
- 5. Enroll a student in a course.
- 6. Schedule a course in a room.
- 7. Record marks.
- 8. View student result.
- 9. Collect fee.
- 10. View classrooms.
- 11. View what each office does.
- 0. Exit.

7. Read the user's choice using nextInt().
8. Call nextLine() to consume the remaining newline.
9. Execute the corresponding case in the switch statement.

16.2 Algorithm of Case 1: View Departments

Step 1: Start a loop from 0 to uni.getDeptCount() - 1.

Step 2: Get the Department object at the current index.

Step 3: Display the department number, department name, and student count.

Step 4: Repeat until all departments have been displayed.

Step 5: Exit the case.

Algorithm

16.3 Algorithm of Case 2: View Courses

Step 1: Start a loop from 0 to uni.getCourseCount() - 1.

Step 2: Get the Course object at the current index.

Step 3: Display the course using its toString() method.

Step 4: Repeat until all courses have been displayed.

Step 5: Exit the case.

16.4 Algorithm of Case 3: View All People

Step 1: Start a loop from 0 to uni.getPeopleCount() - 1.

Step 2: Get the Person object at the current index.

Step 3: Display the object using its toString() method.

Step 4: Repeat until all people have been displayed.

Step 5: Call Person.getTotalPeople().

Step 6: Display the total number of Person objects created.

Step 7: Exit the case.

16.5 Algorithm of Case 4: Admit a New Student

Step 1: Ask the user to enter the student's name.

Step 2: Read the name.

Step 3: Ask the user to enter the student's email.

Step 4: Read the email.

Step 5: Display all departments with their numbers, names, and student counts.

Step 6: Ask the user to select a department number.

Step 7: Read the department number.

Step 8: Check whether the department number is less than 1 or greater than the department count.

Algorithm

Step 9: If the department number is invalid, display "Invalid department." and exit the case.

Step 10: Otherwise, retrieve the selected Department object.

Step 11: Ask the user to select the student level:

- 1 for Undergraduate.
- 2 for Graduate.

Step 12: Read the level.

Step 13: If the level is 1, create an UndergraduateStudent object using the entered name, email, and department.

Step 14: Otherwise, if the level is 2:

1. Ask the user to enter the thesis topic.
2. Read the thesis topic.
3. Create a GraduateStudent object using the entered name, email, department, and thesis topic.

Step 15: If the level is neither 1 nor 2, display "Invalid level." and exit the case.

Step 16: Pass the newly created student to admission.admit(uni, s).

Step 17: Display the result returned by the admission method.

Step 18: Exit the case.

16.6 Algorithm of Case 5: Enroll a Student in a Course

Step 1: Ask the user to enter the student ID.

Step 2: Read the student ID.

Step 3: Call uni.findStudent(sId) to search for the student.

Step 4: If the result is null, display "No student with that ID." and exit the case.

Step 5: Ask the user to enter the course code.

Step 6: Read the course code.

Step 7: Call uni.findCourse(cCode) to search for the course.

Step 8: If the result is null, display "No course with that code." and exit the case.

Algorithm

Step 9: Call targetStudent.enroll(targetCourse).

Step 10: Display the enrollment result.

Step 11: Exit the case.

16.7 Algorithm of Case 6: Schedule a Course in a Room

Step 1: Ask the user to enter the course code.

Step 2: Read the course code.

Step 3: Search for the course using uni.findCourse(schedCode).

Step 4: If the course is not found, display "No course with that code." and exit the case.

Step 5: Display all available room objects with their numbers, room numbers, and capacities.

Step 6: Ask the user to select a room number.

Step 7: Read the room number.

Step 8: Check whether the room number is less than 1 or greater than the room count.

Step 9: If the room number is invalid, display "Invalid room." and exit the case.

Step 10: Otherwise, retrieve the selected Room object.

Step 11: Ask the user to enter the time slot.

Step 12: Read the time slot.

Step 13: Call csStaff.scheduleClass(uni, schedCourse, room, slot).

Step 14: Display the scheduling result.

Step 15: Exit the case.

16.8 Algorithm of Case 7: Record Marks

Step 1: Ask the user to enter the student ID.

Step 2: Read the student ID.

Step 3: Search for the student using uni.findStudent(markStuId).

Step 4: If the student is not found, display "No student with that ID." and exit the case.

Step 5: Ask the user to enter the course code.

Step 6: Read the course code.

Step 7: Search for the course using uni.findCourse(markCourseCode).

Step 8: If the course is not found, display "No course with that code." and exit the case.

Step 9: Ask the user to enter marks between 0 and 100.

Step 10: Read the marks.

Step 11: Call exam.recordMarks(markStudent, markCourse, marksVal).

Step 12: Display the result returned by the method.

Step 13: Exit the case.

16.9 Algorithm of Case 8: View Student Result

Step 1: Ask the user to enter the student ID.

Step 2: Read the student ID.

Step 3: Search for the student using uni.findStudent(resStuId).

Step 4: If the student is not found, display "No student with that ID." and exit the case.

Step 5: Display the student's information using the toString() method.

Step 6: Check whether the student is an instance of GraduateStudent.

Step 7: If true, convert the student object to a GraduateStudent object and display the thesis topic.

Step 8: Start a loop from 0 to resStudent.getCourseCount() - 1.

Step 9: For each enrolled course:

1. Get the course at the current index.
2. Get the corresponding mark.
3. Display the course code and mark.

Step 10: Repeat until all enrolled courses have been displayed.

Step 11: Call getAverage() to calculate the average marks.

Step 12: Call getCourseCount() to obtain the number of enrolled courses.

Step 13: Call getGrade() to calculate the grade.

Step 14: Display the course count, average marks, and grade.

Step 15: Exit the case.

16.10 Algorithm of Case 9: Collect Fee

Step 1: Ask the user to enter the student ID.

Step 2: Read the student ID.

Step 3: Search for the student using uni.findStudent(feeStuId).

Step 4: If the student is not found, display "No student with that ID." and exit the case.

Step 5: Display the student's total fee, fee paid, and fee due.

Step 6: Display the following payment options:

- 1. Pay a part of the fee.
- 2. Pay the full fee due.

Step 7: Ask the user to select a payment option.

Step 8: Read the selected option.

Step 9: If the option is 1:

1. Ask the user to enter the payment amount.
2. Read the amount.
3. Call account.collectFee(feeStudent, amount).
4. Display the result.

Step 10: Otherwise, if the option is 2:

1. Call account.collectFee(feeStudent).
2. Display the result.

Step 11: Otherwise, display "Invalid choice.".

Step 12: Exit the case.

Algorithm

16.11 Algorithm of Case 10: View Classrooms

Step 1: Start a loop from 0 to uni.getRoomCount() - 1.

Step 2: Get the Room object at the current index.

Step 3: Display the room information.

Step 4: Initialize a Boolean variable used to false.

Step 5: Start another loop from 0 to uni.getCourseCount() - 1.

Step 6: Get the Course object at the current index.

Step 7: Check whether the course's room reference is the same as the current Room object.

Step 8: If the references match:

1. Display the course code and time slot.
2. Set used to true.

Step 9: Repeat the inner loop until all courses have been checked.

Step 10: If used is false, display "no classes".

Step 11: Move to the next room and repeat the process.

Step 12: Exit the case.

16.12 Algorithm of Case 11: View Office Duties

Step 1: Create a Staff array containing the following objects:

- Admission Office.
- Account Office.
- Exam Office.
- Departmental Staff.

Step 2: Start a loop from 0 to the length of the Staff array minus 1.

Step 3: For each staff object:

1. Get the office name using getOffice().

Algorithm

2. Get the staff member's name using getName().
3. Call performDuty() to obtain the office's responsibilities.
4. Display the office name, staff member's name, and duty description.

Step 4: Repeat until all four office duties have been displayed.

Step 5: Exit the case.

16.13 Algorithm of Case 0: Exit Program

Step 1: Check whether the user's choice is 0.

Step 2: Display "Goodbye!".

Step 3: Exit the menu loop because the condition choice != 0 is false.

Step 4: Close the Scanner object.

Step 5: End the program.

16.14 Algorithm of Default Case

Step 1: If the user enters a choice that does not match any menu option, execute the default case.

Step 2: Display "Invalid option, try again.".

Step 3: Return to the menu loop.

Step 4: Display the menu again and ask the user to enter another choice.

---

17. End of Algorithm

Step 1: Continue displaying the menu and performing the selected operations until the user enters 0.

Step 2: Close the Scanner object after the menu loop terminates.

Step 3: End the University Management System program.

Algorithm